



Week 04 — OLS Basics

Fitting a regression line by hand and with `lm()`, judging its fit, and log-transforming a skewed predictor

AUTHORS

Johann-Friedrich Salzmänn  

Joshua Elias Schmidt Rodrigues  

DATE

2026-10-02

Topics:

Package management	2
Where we left off	2
The linear regression model	2
Data: German election results (2025)	2
OLS by hand: Greens and FDP	5
Goodness of fit	11
Linear models with <code>lm()</code>	15
Log transformations: Greens and population density	21
Concluding remarks	31
Try it: predicting a county's Greens share	32
Try it: AfD and SPD	32

Package management

Besides the tidyverse, we load `broom` (part of the tidymodels family), which turns model output into tibbles, and `modelsummary` for regression tables.

```
if (!require(pacman)) install.packages("pacman")
```

MESSAGE

Loading required package: pacman

```
p_load(tidyverse, broom, modelsummary)
set.seed(404)
```

Where we left off

Week 3 turned single estimates into confidence intervals and compared AfD support between East and West German counties — one variable, split into two groups. This week we move to the relationship between two continuous variables: how does one party's county-level vote share move with another variable, and how well does a straight line describe that?

The linear regression model

Regression relates a **dependent variable** Y , the quantity we want to explain, to an **independent variable** X we use to explain it. For each observation $i = 1, \dots, n$:

$$Y_i = \beta_0 + \beta_1 X_i + \varepsilon_i$$

β_0 is the **intercept** (the value of Y when $X = 0$), β_1 the **slope** (the change in Y for a one-unit increase in X), and the **error term** ε_i collects everything else that affects Y_i . Estimating β_0 and β_1 from data gives the regression line:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

Ordinary least squares (OLS) picks the line that minimizes the sum of squared residuals $e_i = y_i - \hat{y}_i$:

$$\min_{\hat{\beta}_0, \hat{\beta}_1} \sum_{i=1}^n (y_i - \hat{\beta}_0 - \hat{\beta}_1 x_i)^2$$

To build intuition, drag points around in Setosa's [interactive OLS explainer](#); *Introduction to Econometrics with R* walks through [estimating the coefficients](#) step by step.

Data: German election results (2025)

GERDA again, the county-level federal election results from Week 3, restricted to 2025. We keep the seven parties that entered (or nearly entered) the Bundestag plus turnout, population and area, convert the shares to percent, and add population density (residents per km²):

```
gerda <- read_csv("data/federal_cty_harm.csv")
```

MESSAGE

```
Rows: 4000 Columns: 127
— Column specification —————
Delimiter: ","
chr    (2): county_code, state
dbl   (124): election_year, eligible_voters, number_voters, valid_votes, inva...
date   (1): election_date

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
counties <- gerda |>
  filter(election_year == 2025) |>
  mutate(density = population / area) |>
  select(
    county_code, state,
    cdu_csu, spd, greens = gruene, fdp, linke = linke_pds, afd, bsw,
    turnout, population, area, density
  ) |>
  mutate(across(cdu_csu:turnout, ~ .x * 100))
counties
```

OUTPUT

```
# A tibble: 400 × 13
  county_code state cdu_csu  spd greens  fdp linke  afd  bsw turnout
  <chr>      <chr>   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1 01001      01      17.4  15.7  20.6  3.30  12.9  13.8  3.33  79.4
2 01002      01      18.7  19.3  23.1  3.83  14.4  10.9  3.36  82.5
3 01003      01      21.9  21.4  18.3  3.79  10.9  15.3  3.78  79.5
4 01004      01      25.3  19.6  10.5  4.14  8.06  22.3  4.62  76.2
5 01051      01      30.3  17.7   9.15  5.51  6.10  21.1  3.51  82.1
6 01053      01      28.8  18.5  13.2  5.07  7.12  19.1  3.80  84.1
7 01054      01      29.1  17.3  13.4  4.80  6.17  14.5  3.04  83.2
8 01055      01      31.2  19.9  12.6  5.16  5.92  17.6  3.09  82.9
9 01056      01      28.9  20.2  14.5  5.06  7.57  15.6  3.57  84.6
10 01057     01      29.9  19.2  16.0  4.32  6.27  15.7  3.03  86.2
# i 390 more rows
# i 3 more variables: population <dbl>, area <dbl>, density <dbl>
```

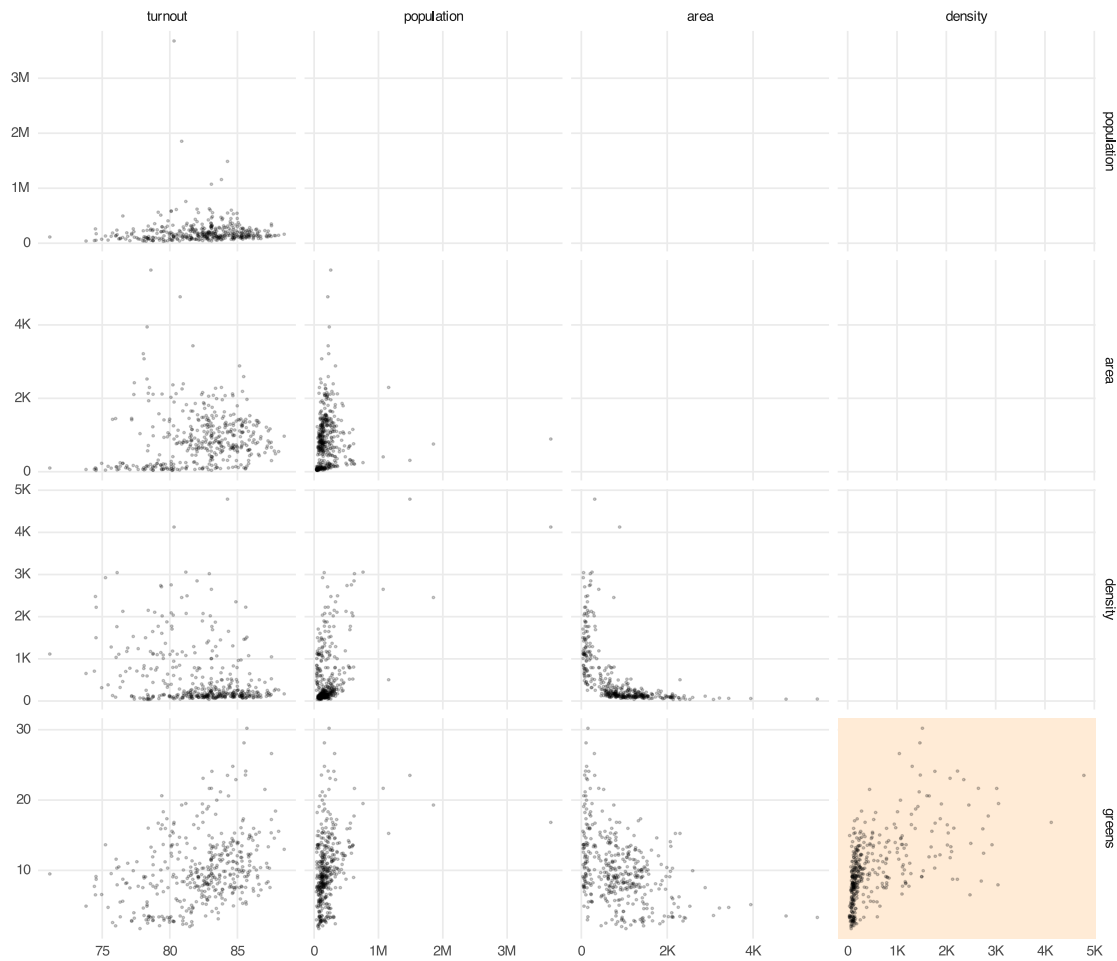
Before modelling anything, look at all 21 pairings of parties at once, one scatter plot per pair of parties:



Some pairings form a single tight cloud, some barely relate at all, and some split into separate clusters. Soon we'll fit a straight line through one of these clouds by hand — Greens and FDP, highlighted.

The same matrix for the county covariates — turnout, population, area and density — with the Greens share in the bottom row. The highlighted Greens-density panel comes back later in the lab:

Greens share and county covariates, 2025



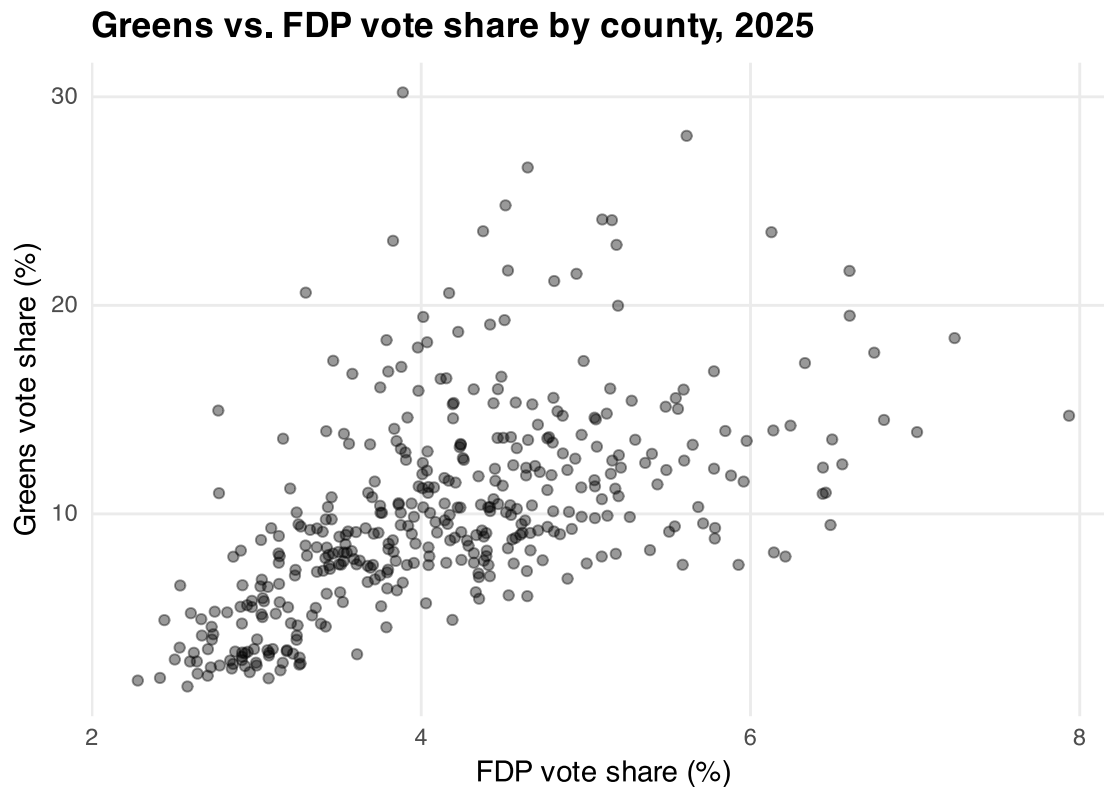
Population, area and density bunch up at the low end with a few extreme counties — a skew we come back to in the log-transformation section.

OLS by hand: Greens and FDP

Greens and FDP compete for a similar electorate — well-educated, higher income, often urban. Do the Greens do better in counties where the FDP does better? Greens share is our y , FDP share our x , so the model is:

$$\text{Greens}_i = \beta_0 + \beta_1 \text{FDP}_i + \varepsilon_i$$

```
counties |>
  ggplot(aes(x = fdp, y = greens)) +
  geom_point(alpha = 0.4) +
  labs(
    title = "Greens vs. FDP vote share by county, 2025",
    x = "FDP vote share (%)",
    y = "Greens vote share (%)"
  ) +
  theme_intror()
```



Slope and intercept

The OLS slope is the sample covariance of x and y , scaled by the sample variance of x :

$$\hat{\beta}_1 = \frac{\text{Cov}(x, y)}{\text{Var}(x)}$$

```
counties |>
  summarize(beta_1 = cov(fdp, greens) / var(fdp))
```

> OUTPUT

```
# A tibble: 1 × 1
  beta_1
  <dbl>
1    2.75
```

Both Cov and Var divide by $n - 1$, which cancels. Written out, the slope is a ratio of two sums of deviations from the means:

$$\hat{\beta}_1 = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

We build it step by step, starting with the two means:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

```
mean_x <- counties |>
  summarize(mean_x = mean(fdp)) |>
  pull(mean_x)
mean_x
```

> OUTPUT

```
[1] 4.13741
```

$$\bar{y} = \frac{1}{n} \sum_{i=1}^n y_i$$

```
mean_y <- counties |>
  summarize(mean_y = mean(greens)) |>
  pull(mean_y)
mean_y
```

> OUTPUT

```
[1] 10.05147
```

Per county, the deviations from the means, their product, and the squared x deviation:

$$x_i - \bar{x}$$

$$y_i - \bar{y}$$

$$(x_i - \bar{x})(y_i - \bar{y})$$

$$(x_i - \bar{x})^2$$

```
ols_hand <- counties |>
  select(county_code, fdp, greens) |>
  mutate(
    diff_x      = fdp - mean_x,
    diff_y      = greens - mean_y,
    diff_xy     = diff_x * diff_y,
    diff_x_squared = diff_x^2
  )
ols_hand
```

> OUTPUT

```
# A tibble: 400 × 7
  county_code fdp greens diff_x diff_y diff_xy diff_x_squared
  <chr>      <dbl> <dbl>   <dbl> <dbl>   <dbl>         <dbl>
1 01001      3.30  20.6 -0.839  10.6   -8.86          0.704
2 01002      3.83  23.1 -0.309  13.0   -4.02          0.0952
3 01003      3.79  18.3 -0.348   8.28   -2.88          0.121
4 01004      4.14  10.5  0.00464  0.447  0.00207       0.0000216
5 01051      5.51   9.15  1.37   -0.906 -1.24          1.87
6 01053      5.07  13.2  0.929   3.17   2.95          0.864
7 01054      4.80  13.4  0.661   3.37   2.23          0.437
8 01055      5.16  12.6  1.02    2.50   2.56          1.05
9 01056      5.06  14.5  0.923   4.48   4.14          0.853
10 01057      4.32  16.0  0.181   5.92   1.07          0.0329
# i 390 more rows
```

Summing the last two columns over all counties gives the numerator and the denominator:

$$\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

$$\sum_{i=1}^n (x_i - \bar{x})^2$$

```
ols_sums <- ols_hand |>
  summarize(
    numerator = sum(diff_xy),
    denominator = sum(diff_x_squared)
  )
ols_sums
```

> OUTPUT

```
# A tibble: 1 × 2
  numerator denominator
  <dbl>         <dbl>
1  1060.         385.
```

```

numerator <- ols_sums |>
  pull(numerator)
denominator <- ols_sums |>
  pull(denominator)

```

Their ratio is the slope — the same value `cov()` and `var()` gave:

$$\hat{\beta}_1 = \frac{\text{numerator}}{\text{denominator}}$$

```

beta_1 <- numerator / denominator
beta_1

```

> OUTPUT

```
[1] 2.752281
```

The intercept then makes the line pass through the point of means $(\bar{x}, \bar{y})$, reusing both means and the slope:

$$\hat{\beta}_0 = \bar{y} - \hat{\beta}_1 \bar{x}$$

```

beta_0 <- mean_y - beta_1 * mean_x
beta_0

```

> OUTPUT

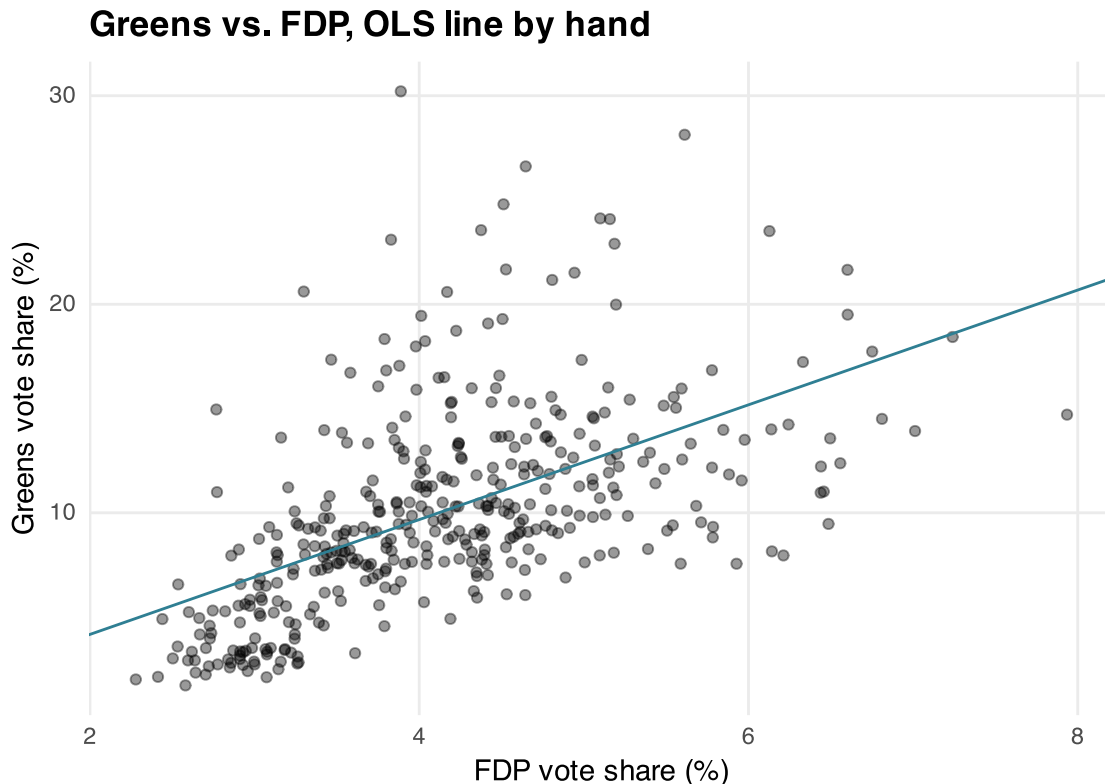
```
[1] -1.335843
```

Filled in with both estimates, the fitted line is:

$$\widehat{\text{Greens}}_i = -1.34 + 2.75 \cdot \text{FDP}_i$$

`geom_abline()` draws a line from an intercept and a slope — here, the two numbers we just computed:

```
counties |>
  ggplot(aes(x = fdp, y = greens)) +
  geom_point(alpha = 0.4) +
  geom_abline(intercept = beta_0, slope = beta_1, color = course_primary) +
  labs(
    title = "Greens vs. FDP, OLS line by hand",
    x     = "FDP vote share (%)",
    y     = "Greens vote share (%)"
  ) +
  theme_intror()
```



On average, a county with one percentage point more FDP support has 2.75 points more Greens support. The intercept, -1.34 , is the predicted Greens share at 0% FDP — an impossible negative share, and far outside the data (no county has the FDP below 2%). Intercepts are often not meaningful on their own.

Fitted values and residuals

Each county's **fitted value** is the line's prediction at its own FDP share, and its **residual** is what the line misses:

$$\hat{y}_i = \hat{\beta}_0 + \hat{\beta}_1 x_i$$

$$e_i = y_i - \hat{y}_i$$

We add both as columns to `ols_hand`:

```
ols_hand <- ols_hand |>
  mutate(
    greens_hat = beta_0 + beta_1 * fdp,
    residual = greens - greens_hat
  )
ols_hand
```

> OUTPUT

```
# A tibble: 400 × 9
  county_code fdp greens diff_x diff_y diff_xy diff_x_squared greens_hat
  <chr>      <dbl> <dbl>   <dbl> <dbl>   <dbl>      <dbl>      <dbl>
1 01001      3.30  20.6  -0.839  10.6   -8.86         0.704       7.74
2 01002      3.83  23.1  -0.309  13.0   -4.02         0.0952      9.20
3 01003      3.79  18.3  -0.348   8.28  -2.88         0.121       9.09
4 01004      4.14  10.5   0.00464 0.447  0.00207      0.0000216    10.1
5 01051      5.51   9.15   1.37   -0.906 -1.24         1.87       13.8
6 01053      5.07  13.2   0.929   3.17   2.95         0.864       12.6
7 01054      4.80  13.4   0.661   3.37   2.23         0.437       11.9
8 01055      5.16  12.6   1.02    2.50   2.56         1.05       12.9
9 01056      5.06  14.5   0.923   4.48   4.14         0.853       12.6
10 01057      4.32  16.0   0.181   5.92   1.07         0.0329      10.6
# i 390 more rows
# i 1 more variable: residual <dbl>
```

By construction, OLS residuals sum to zero (up to floating-point noise):

$$\sum_{i=1}^n e_i = 0$$

```
ols_hand |>
  summarize(sum_residuals = sum(residual))
```

> OUTPUT

```
# A tibble: 1 × 1
  sum_residuals
  <dbl>
1 0.000000000000000888
```

Goodness of fit

How much of the variation in Greens support does the line explain? Every county's distance from the mean $\bar{y}$ splits into a part the line explains and a residual part. Per county, we add the explained deviation and the squares of all three parts to `ols_hand` — the total deviation $y_i - \bar{y}$ (`diff_y`) and the residual e_i are already there:

$$(y_i - \bar{y})^2$$

$$\hat{y}_i - \bar{y}$$

$$(\hat{y}_i - \bar{y})^2$$

$$e_i^2$$

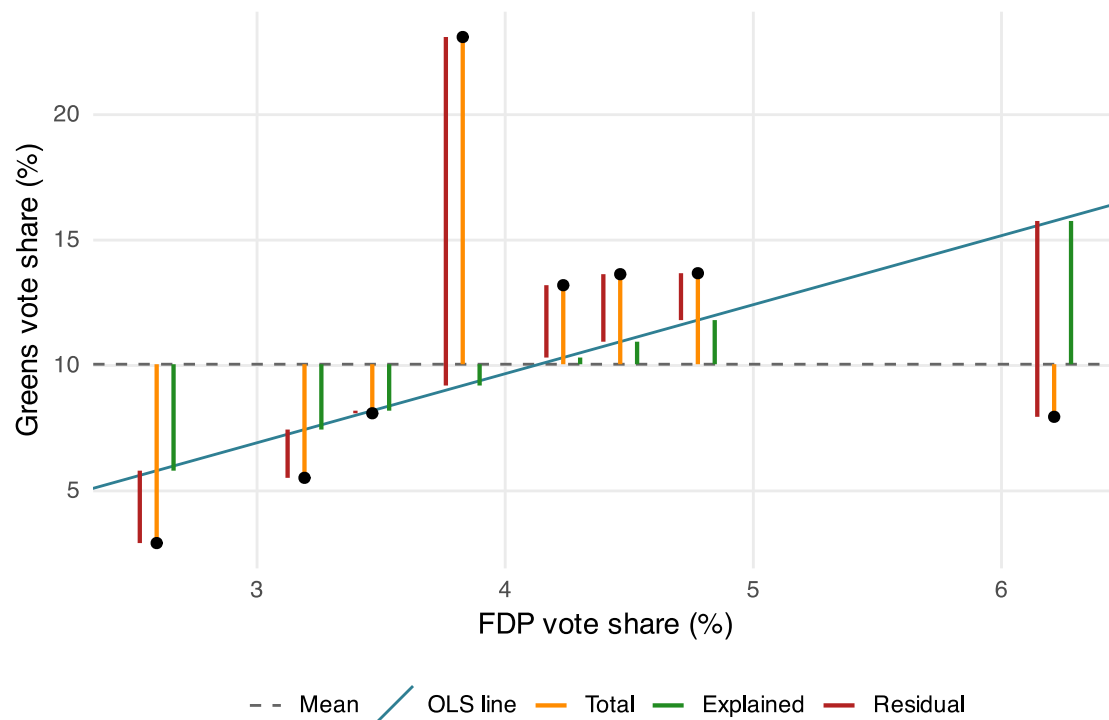
```
ols_hand <- ols_hand |>
  mutate(
    diff_y_squared = diff_y^2,
    diff_hat       = greens_hat - mean_y,
    diff_hat_squared = diff_hat^2,
    residual_squared = residual^2
  )
ols_hand
```

> OUTPUT

```
# A tibble: 400 × 13
  county_code  fdp greens  diff_x diff_y diff_xy diff_x_squared greens_hat
  <chr>      <dbl> <dbl>   <dbl> <dbl>   <dbl>       <dbl>      <dbl>
1 01001      3.30  20.6 -0.839  10.6   -8.86         0.704       7.74
2 01002      3.83  23.1 -0.309  13.0   -4.02         0.0952      9.20
3 01003      3.79  18.3 -0.348   8.28  -2.88         0.121       9.09
4 01004      4.14  10.5  0.00464 0.447  0.00207      0.0000216    10.1
5 01051      5.51   9.15  1.37   -0.906 -1.24         1.87       13.8
6 01053      5.07  13.2  0.929   3.17   2.95         0.864      12.6
7 01054      4.80  13.4  0.661   3.37   2.23         0.437      11.9
8 01055      5.16  12.6  1.02   2.50   2.56         1.05      12.9
9 01056      5.06  14.5  0.923   4.48   4.14         0.853      12.6
10 01057     4.32  16.0  0.181   5.92   1.07         0.0329     10.6
# i 390 more rows
# i 5 more variables: residual <dbl>, diff_y_squared <dbl>, diff_hat <dbl>,
#   diff_hat_squared <dbl>, residual_squared <dbl>
```

For eight counties spread across the range of FDP shares, the three parts as segments, side by side: the total deviation from the mean to the county (centre), the residual from the line to the county (left), and the explained part from the mean up to the line (right):

Splitting up the variation in Greens support



Summing each squared column over all counties gives three sums of squares. The **total sum of squares** is the variation in y itself, the **explained sum of squares** the variation the line reproduces, and the **residual sum of squares** what's left over — the quantity OLS minimized:

$$\text{TSS} = \sum_{i=1}^n (y_i - \bar{y})^2$$

$$\text{ESS} = \sum_{i=1}^n (\hat{y}_i - \bar{y})^2$$

$$\text{RSS} = \sum_{i=1}^n e_i^2$$

```
ss_sums <- ols_hand |>
  summarize(
    tss = sum(diff_y_squared),
    ess = sum(diff_hat_squared),
    rss = sum(residual_squared)
  )
ss_sums
```

> OUTPUT

```
# A tibble: 1 × 3
  tss  ess  rss
<dbl> <dbl> <dbl>
1 9287. 2916. 6370.
```

```
tss <- ss_sums |>
  pull(tss)
ess <- ss_sums |>
  pull(ess)
rss <- ss_sums |>
  pull(rss)
```

The two parts add back up to the total:

$$\text{TSS} = \text{ESS} + \text{RSS}$$

```
c(tss = tss, ess_plus_rss = ess + rss)
```

> OUTPUT

```
      tss ess_plus_rss
9286.69    9286.69
```

R^2 is the share of the total variation the model explains, bounded between 0 and 1:

$$R^2 = \frac{\text{ESS}}{\text{TSS}}$$

```
r_squared <- ess / tss
r_squared
```

> OUTPUT

```
[1] 0.3140497
```

Equivalently, one minus the unexplained share:

$$R^2 = 1 - \frac{\text{RSS}}{\text{TSS}}$$

```
1 - rss / tss
```

> OUTPUT

```
[1] 0.3140497
```

The **correlation coefficient** r_{xy} measures the strength and direction of a linear relationship on a scale from -1 to 1 . With a single predictor, R^2 is just the squared correlation, so r_{xy} is its square root — positive here, since it takes the sign of the slope:

$$r_{xy} = \sqrt{R^2}$$

```
r <- sqrt(r_squared)
r
```

> OUTPUT

```
[1] 0.5604014
```

`cor()` gives the same value:

```
counties |>
  summarize(r = cor(fdp, greens))
```

> OUTPUT

```
# A tibble: 1 × 1
      r
  <dbl>
1 0.560
```

All quantities at a glance, with the R object each one is stored in:

Quantity	Formula	R object	Explanation
Residual	$e_i = y_i - \hat{y}_i$	<code>residual</code> (column of <code>ols_hand</code>)	What the line misses for one county
TSS	$\sum_{i=1}^n (y_i - \bar{y})^2$	<code>tss</code>	Total variation of y around its mean
ESS	$\sum_{i=1}^n (\hat{y}_i - \bar{y})^2$	<code>ess</code>	Variation the line reproduces
RSS	$\sum_{i=1}^n e_i^2$	<code>rss</code>	Variation left over, minimized by OLS
R^2	$\frac{ESS}{TSS} = 1 - \frac{RSS}{TSS}$	<code>r_squared</code>	Share of the variation explained, between 0 and 1
r_{xy}	$\sqrt{R^2}$ (sign of $\hat{\beta}_1$)	<code>r</code>	Strength and direction of the linear relationship, between -1 and 1

The FDP share accounts for about a third of the county-level variation in Greens support — a real relationship, but most of the variation lies elsewhere.

Linear models with `lm()`

R fits all of the above in one call. `lm()` takes a formula, `outcome ~ predictor`, which mirrors the model equation (the intercept is included automatically):

$$\text{Greens}_i = \beta_0 + \beta_1 \text{FDP}_i + \varepsilon_i$$

```
fdp_model <- lm(greens ~ fdp, data = counties)
fdp_model
```

> OUTPUT

```
Call:
lm(formula = greens ~ fdp, data = counties)

Coefficients:
(Intercept)      fdp
    -1.336      2.752
```

Base R's classic way to inspect a model is `summary()`. It prints everything at once, as text rather than a table — the standard errors, t-values and p-values in it are Week 6 material:

```
summary(fdp_model)
```

> OUTPUT

```
Call:
lm(formula = greens ~ fdp, data = counties)

Residuals:
    Min       1Q   Median       3Q      Max
-7.807 -2.612 -0.710  1.569 20.845

Coefficients:
              Estimate Std. Error t value      Pr(>|t|)
(Intercept)  -1.3358     0.8670  -1.541      0.124
fdp           2.7523     0.2039  13.499 <0.0000000000000002 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 4.001 on 398 degrees of freedom
Multiple R-squared:  0.314, Adjusted R-squared:  0.3123
F-statistic: 182.2 on 1 and 398 DF, p-value: < 0.00000000000000022
```

From here on we stay with `broom`, which returns each part of a model as a tibble. `tidy()` gives one row per coefficient:

```
fdp_model |>
  tidy()
```

> OUTPUT

```
# A tibble: 2 × 5
  term      estimate std.error statistic    p.value
<chr>    <dbl>    <dbl>    <dbl>    <dbl>
1 (Intercept) -1.34    0.867    -1.54 1.24e- 1
2 fdp         2.75    0.204    13.5  1.88e-34
```

Pulled out of that tibble, the two estimates fill in the fitted line:

```
lm_beta_0 <- fdp_model |>
  tidy() |>
  filter(term == "(Intercept)") |>
  pull(estimate)
lm_beta_1 <- fdp_model |>
  tidy() |>
  filter(term == "fdp") |>
  pull(estimate)
lm_beta_0
```

> OUTPUT

```
[1] -1.335843
```

```
lm_beta_1
```

> OUTPUT

```
[1] 2.752281
```

$$\widehat{\text{Greens}}_i = -1.34 + 2.75 \cdot \text{FDP}_i$$

The estimates match our hand-computed ones exactly:

```
coef_comparison <- tibble(
  term      = c("(Intercept)", "fdp"),
  by_hand   = c(beta_0, beta_1)
) |>
  left_join(tidy(fdp_model), by = "term") |>
  select(term, by_hand, estimate)
coef_comparison
```

> OUTPUT

```
# A tibble: 2 × 3
  term      by_hand estimate
<chr>      <dbl>    <dbl>
1 (Intercept) -1.34    -1.34
2 fdp         2.75     2.75
```

`glance()` gives one row of model-level statistics, including R^2 . Its `adj.r.squared` column penalizes R^2 for the number of predictors k — this only starts to matter once models have several predictors (Week 5):

$$R^2_{\text{adj}} = 1 - (1 - R^2) \frac{n - 1}{n - k - 1}$$

```
fdp_model |>
  glance()
```

> OUTPUT

```
# A tibble: 1 × 12
  r.squared adj.r.squared sigma statistic      p.value    df logLik   AIC   BIC
    <dbl>      <dbl> <dbl>    <dbl>      <dbl> <dbl> <dbl> <dbl> <dbl>
1    0.314      0.312  4.00    182.      1.88e-34     1 -1121. 2248. 2260.
# i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

```
n_counties <- nrow(counties)
1 - (1 - r_squared) * (n_counties - 1) / (n_counties - 1 - 1)
```

> OUTPUT

```
[1] 0.3123263
```

`augment()` adds the fitted values (`.fitted`) and residuals (`.resid`) as columns next to the data used for fitting — the same values as `greens_hat` and `residual` above:

```
fdp_fit <- fdp_model |>
  augment()
fdp_fit
```

> OUTPUT

```
# A tibble: 400 × 8
  greens fdp .fitted .resid .hat .sigma .cooksd .std.resid
    <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
1  20.6  3.30   7.74 12.9  0.00433  3.95 0.0226   3.22
2  23.1  3.83   9.20 13.9  0.00275  3.94 0.0167   3.48
3  18.3  3.79   9.09  9.24  0.00281  3.98 0.00755   2.31
4  10.5  4.14  10.1  0.434 0.00250  4.01 0.0000148  0.109
5   9.15 5.51  13.8 -4.67  0.00736  4.00 0.00510  -1.17
6  13.2  5.07  12.6  0.614 0.00474  4.01 0.0000565   0.154
7  13.4  4.80  11.9  1.55  0.00363  4.00 0.000275   0.388
8  12.6  5.16  12.9 -0.311 0.00522  4.01 0.0000159 -0.0779
9  14.5  5.06  12.6  1.94  0.00471  4.00 0.000560   0.486
10 16.0  4.32  10.6  5.43  0.00259  4.00 0.00239   1.36
# i 390 more rows
```

```
fdp_fit |>
  summarize(rss = sum(.resid^2))
```

> OUTPUT

```
# A tibble: 1 × 1
  rss
  <dbl>
1 6370.
```

With `newdata`, `augment()` applies the fitted line to any tibble that has an `fdp` column — here a random sample of 15 counties. Since these rows still carry their actual `greens` share, residuals come along too:

```
fdp_model |>
  augment(newdata = counties |> slice_sample(n = 15)) |>
  select(county_code, fdp, greens, .fitted, .resid)
```

> OUTPUT

```
# A tibble: 15 × 5
  county_code fdp greens .fitted .resid
  <chr>      <dbl> <dbl> <dbl> <dbl>
1 03157      3.56  9.16  8.45  0.707
2 03459      4.22 10.3  10.3  0.0228
3 03356      3.91 12.6  9.42  3.17
4 03357      3.96  9.86  9.55  0.309
5 05916      2.86  7.95  6.53  1.42
6 06632      3.85  6.33  9.27 -2.93
7 09777      3.88 10.1  9.33  0.728
8 06413      4.46 13.6 10.9   2.69
9 09562      4.38 23.6 10.7  12.8
10 16054      2.84  2.97  6.48 -3.51
11 10043      3.76  5.57  9.00 -3.43
12 09272      3.24  4.16  7.59 -3.43
13 09561      3.20 11.2  7.48  3.73
14 09174      4.69 12.3 11.6  0.720
15 15081      3.07  3.46  7.10 -3.64
```

Regression tables

Papers report models as regression tables. `modelsummary()` builds one from a fitted model: `coef_map` renames (and selects) the coefficients, `gof_map` picks the fit statistics, and `statistic = NULL` and `stars = FALSE` leave out the standard errors and significance stars for now (Week 6):

```

modelsummary(
  fdp_model,
  output    = "tinytable",
  coef_map  = c(
    "(Intercept)" = "Intercept",
    "fdp"          = "FDP vote share (%)"
  ),
  statistic = NULL,
  stars     = FALSE,
  gof_map   = c("nobs", "r.squared"),
  fmt       = 3
)

```

	(1)
Intercept	-1.336
FDP vote share (%)	2.752
Num.Obs.	400
R2	0.314

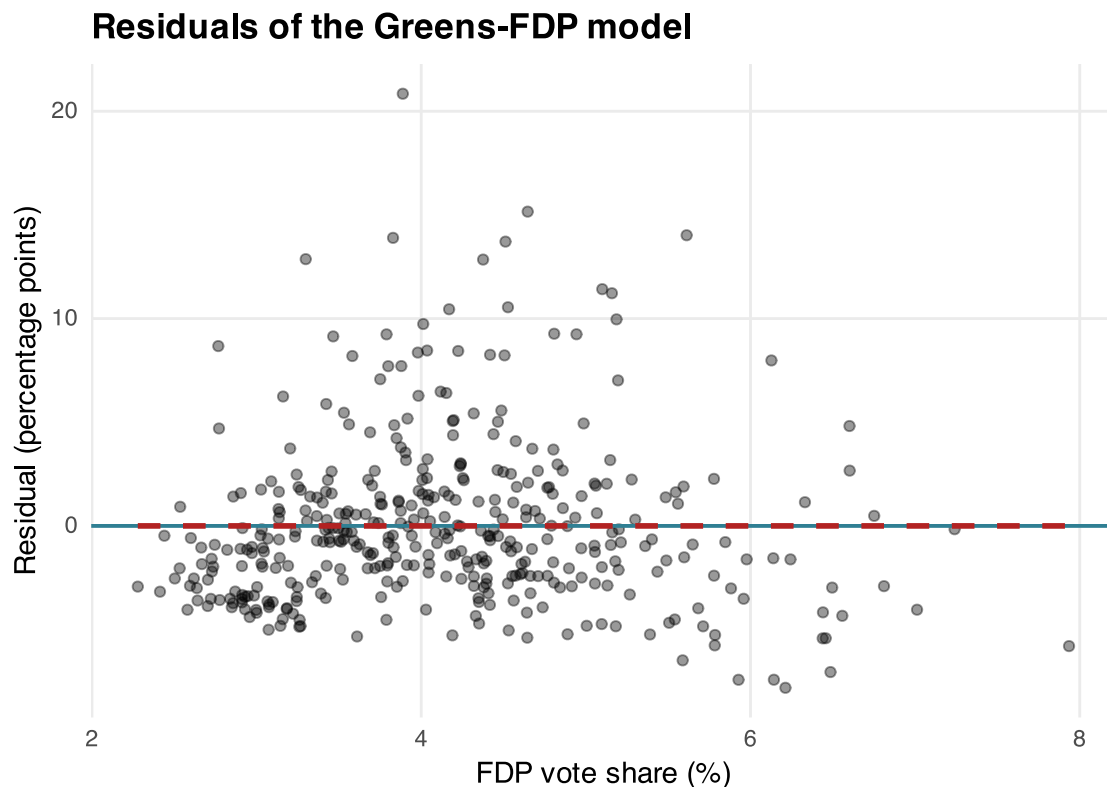
With `output = "tinytable"`, the table adapts to whatever format the document is rendered to — an HTML table here, a Typst table in the PDF, same code. See the [modelsummary documentation](#) for everything else it can do.

Residual plot

Plotting residuals against x shows what the line systematically misses. Ideally they scatter evenly around zero, like pure noise:

A dashed OLS line fitted through the residuals themselves (`geom_smooth()`) lies exactly on the zero line: OLS residuals are uncorrelated with x by construction.

```
fdp_fit |>
  ggplot(aes(x = fdp, y = .resid)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, color = "firebrick", linewidth = 0.8) +
  geom_smooth(
    method = "lm",
    formula = y ~ x,
    se = FALSE,
    color = "firebrick",
    linetype = "dashed"
  ) +
  labs(
    title = "Residuals of the Greens-FDP model",
    x = "FDP vote share (%)",
    y = "Residual (percentage points)"
  ) +
  theme_intror()
```

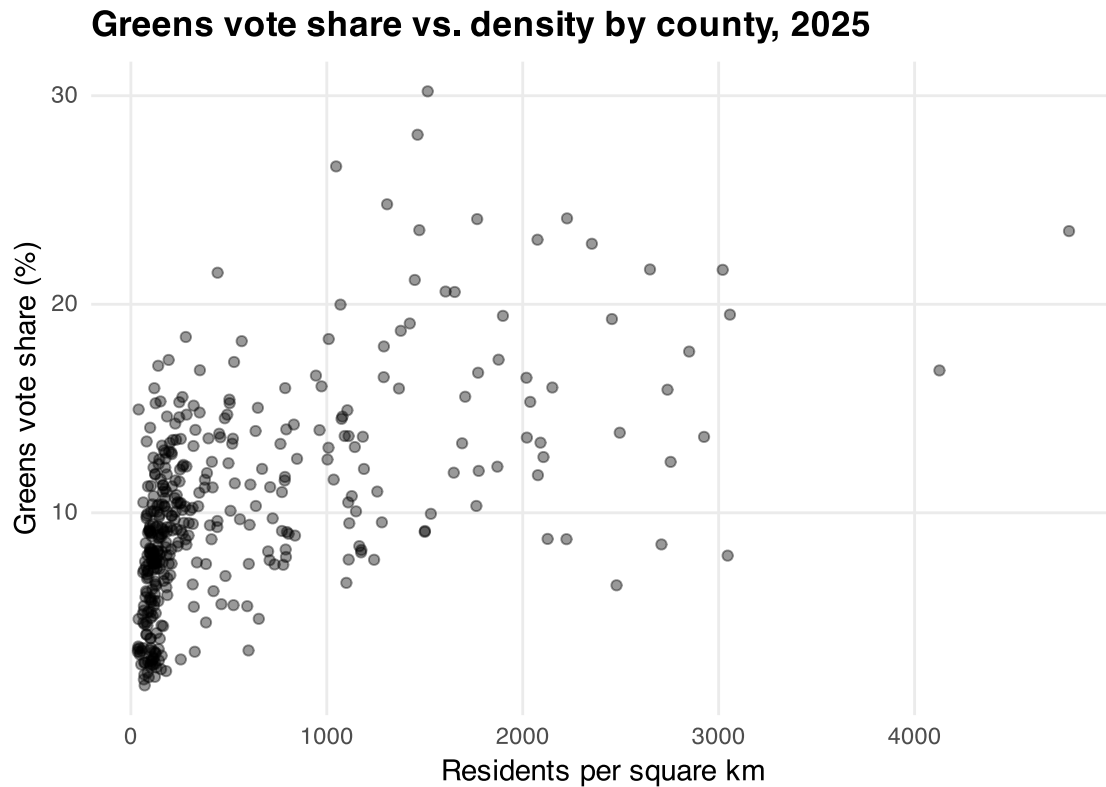


Not quite noise: residuals are mostly negative at both ends of the FDP range and positive in the middle — the relationship bends, which a straight line can't follow.

Log transformations: Greens and population density

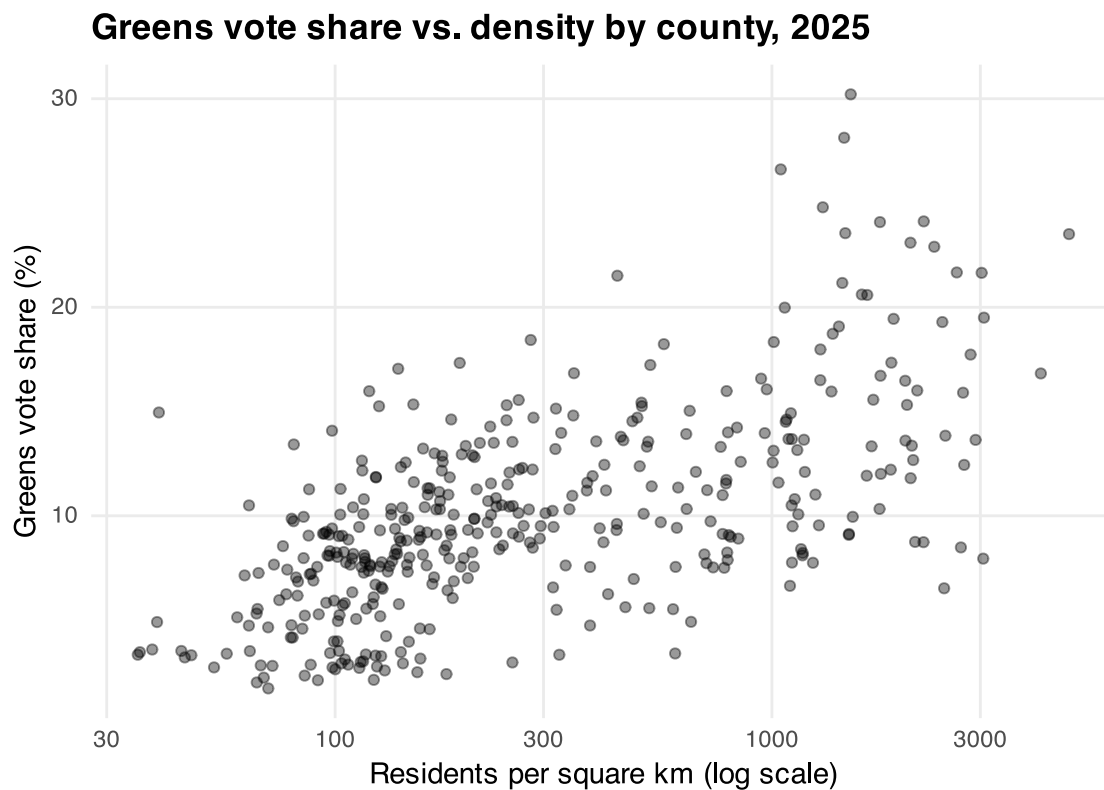
The Greens are often called a party of the cities. Population density is a natural way to measure how urban a county is — but it is strongly right-skewed, which a straight line handles badly. Greens share is again our y , density our x :

```
counties |>
  ggplot(aes(x = density, y = greens)) +
  geom_point(alpha = 0.4) +
  scale_x_continuous(breaks = seq(0, 4000, 1000)) +
  labs(
    title = "Greens vote share vs. density by county, 2025",
    x     = "Residents per square km",
    y     = "Greens vote share (%)"
  ) +
  theme_intror()
```



Put the density axis on a log scale, and the rural counties crowded at the left spread out: the Greens share now rises roughly steadily with density, if with a wide scatter:

```
counties |>
  ggplot(aes(x = density, y = greens)) +
  geom_point(alpha = 0.4) +
  scale_x_log10() +
  labs(
    title = "Greens vote share vs. density by county, 2025",
    x     = "Residents per square km (log scale)",
    y     = "Greens vote share (%)"
  ) +
  theme_intror()
```



Distribution of population density

Most counties are sparsely populated; a few cities are far denser than everyone else. The natural logarithm, $\log(x)$, pulls that long right tail in:

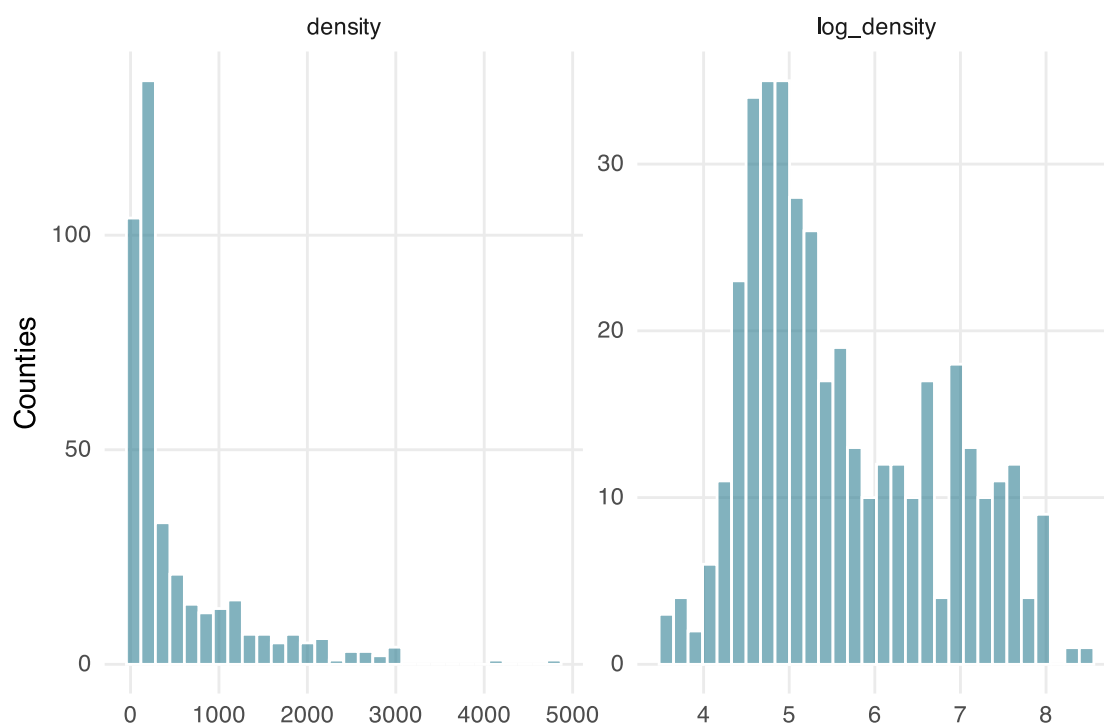
```
density_scales <- counties |>
  mutate(log_density = log(density)) |>
  select(county_code, density, log_density) |>
  pivot_longer(c(density, log_density), names_to = "scale", values_to = "value")
density_scales
```

> OUTPUT

```
# A tibble: 800 × 3
  county_code scale      value
  <chr>      <chr>    <dbl>
1 01001     density  1606.
2 01001   log_density   7.38
3 01002     density  2075.
4 01002   log_density   7.64
5 01003     density  1010.
6 01003   log_density   6.92
7 01004     density  1109.
8 01004   log_density   7.01
9 01051     density   93.8
10 01051   log_density   4.54
# i 790 more rows
```

```
density_scales |>
  ggplot(aes(x = value)) +
  geom_histogram(bins = 30, fill = course_primary, alpha = 0.6, color = "white") +
  facet_wrap(~ scale, scales = "free") +
  labs(
    title = "Population density, raw and logged",
    x      = NULL,
    y      = "Counties"
  ) +
  theme_intror()
```

Population density, raw and logged



Linear and log models

“Linear” regression means linear in the *parameters* β_0, β_1 , not in the variables — we’re free to transform x first. First, the untransformed model:

$$\text{Greens}_i = \beta_0 + \beta_1 \text{Density}_i + \varepsilon_i$$

```
density_model <- lm(greens ~ density, data = counties)
density_model |>
  tidy()
```

➤ OUTPUT

```
# A tibble: 2 × 5
  term      estimate std.error statistic    p.value
<chr>      <dbl>    <dbl>    <dbl>    <dbl>
1 (Intercept)  8.03      0.252     31.8 2.13e-111
2 density    0.00377  0.000284    13.3 1.41e- 33
```

```
density_beta_0 <- density_model |>
  tidy() |>
  filter(term == "(Intercept)") |>
  pull(estimate)
density_beta_1 <- density_model |>
  tidy() |>
  filter(term == "density") |>
  pull(estimate)
```

$$\widehat{\text{Greens}}_i = 8.03 + 0.00377 \cdot \text{Density}_i$$

Then the same model on logged density — `log()` can go straight into the `lm()` formula:

$$\text{Greens}_i = \beta_0 + \beta_1 \log(\text{Density}_i) + \varepsilon_i$$

```
log_density_model <- lm(greens ~ log(density), data = counties)
log_density_model |>
  tidy()
```

> OUTPUT

```
# A tibble: 2 × 5
  term          estimate std.error statistic    p.value
<chr>         <dbl>     <dbl>     <dbl>    <dbl>
1 (Intercept)   -5.29      0.977     -5.42  1.06e- 7
2 log(density)    2.73      0.170     16.0   7.25e-45
```

```
log_beta_0 <- log_density_model |>
  tidy() |>
  filter(term == "(Intercept)") |>
  pull(estimate)
log_beta_1 <- log_density_model |>
  tidy() |>
  filter(term == "log(density)") |>
  pull(estimate)
```

$$\widehat{\text{Greens}}_i = -5.29 + 2.73 \cdot \log(\text{Density}_i)$$

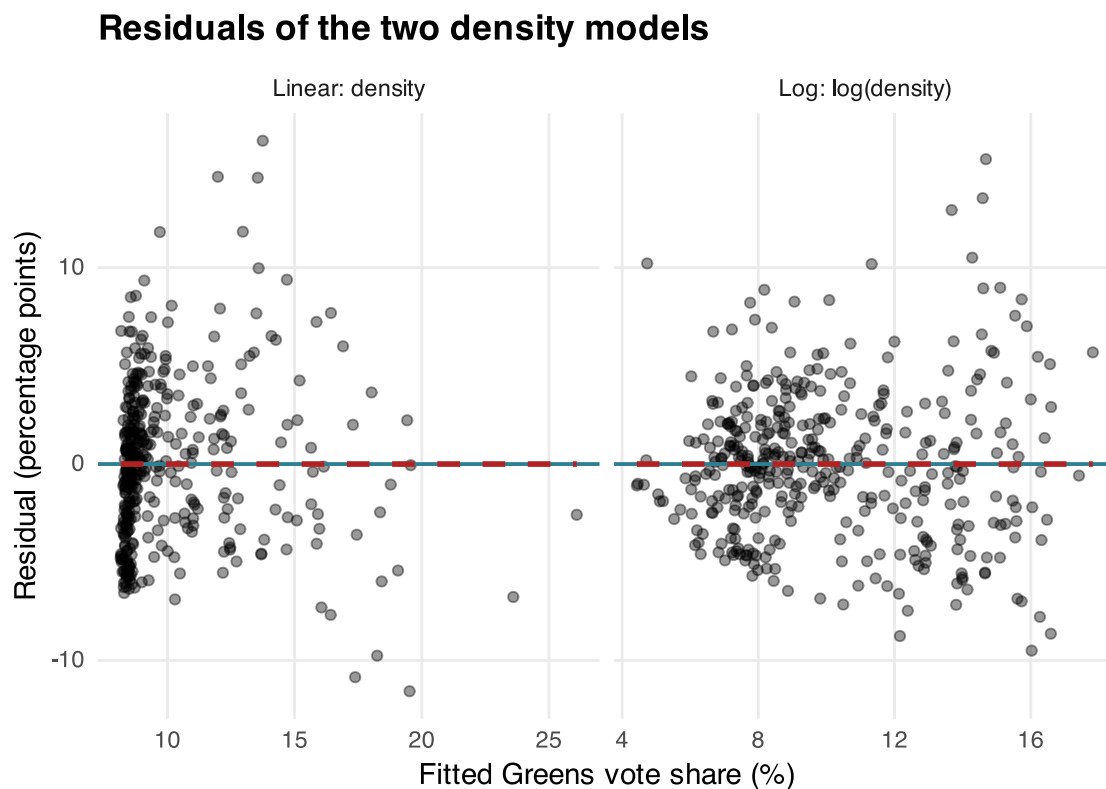
`augment(data = counties)` keeps every original column next to the fitted values, so both models can be compared on the same raw `density` axis:

```
density_fit <- density_model |>
  augment(data = counties) |>
  mutate(model = "Linear: density")
log_density_fit <- log_density_model |>
  augment(data = counties) |>
  mutate(model = "Log: log(density)")

density_fits <- bind_rows(density_fit, log_density_fit)
```

As before, a dashed OLS line through the residuals lies on zero — here plotted against the fitted values, which are just a linear function of x :

```
density_fits |>
  ggplot(aes(x = .fitted, y = .resid)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, color = course_primary, linewidth = 0.8) +
  geom_smooth(
    method = "lm",
    formula = y ~ x,
    se = FALSE,
    color = "firebrick",
    linetype = "dashed"
  ) +
  facet_wrap(~ model, scales = "free_x") +
  labs(
    title = "Residuals of the two density models",
    x = "Fitted Greens vote share (%)",
    y = "Residual (percentage points)"
  ) +
  theme_intror()
```



The linear model gives almost every rural county nearly the same prediction (around 8%), although their actual shares range widely — the cloud of residuals piles up on the left. The log model spreads its predictions out and leaves a much more even band. The fit statistics agree:

```

density_glance <- density_model |>
  glance() |>
  mutate(model = "Linear: density")
log_density_glance <- log_density_model |>
  glance() |>
  mutate(model = "Log: log(density)")

bind_rows(density_glance, log_density_glance) |>
  select(model, r.squared, adj.r.squared)

```

➤ OUTPUT

```

# A tibble: 2 × 3
  model          r.squared adj.r.squared
  <chr>          <dbl>      <dbl>
1 Linear: density    0.307      0.305
2 Log: log(density)  0.392      0.390

```

`modelsummary()` also takes a named list of models and sets them side by side as columns, one per name:

```

density_models <- list(
  "Linear" = density_model,
  "Log"    = log_density_model
)

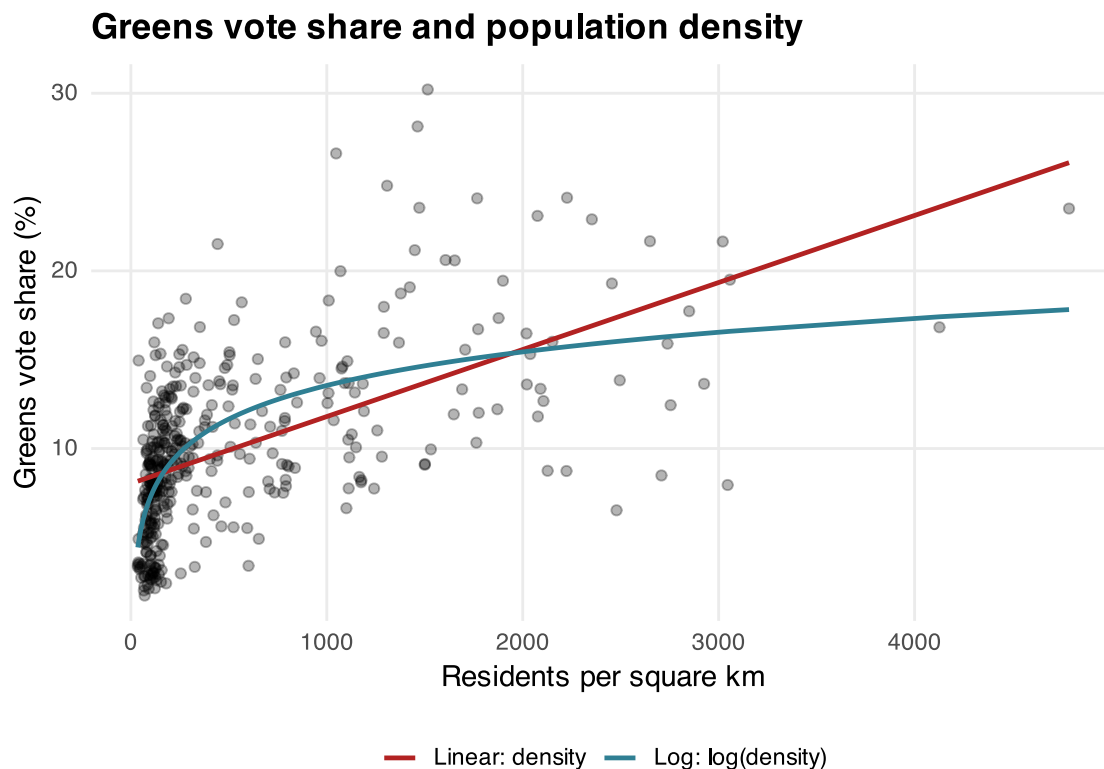
modelsummary(
  density_models,
  output      = "tinytable",
  coef_map    = c(
    "(Intercept)" = "Intercept",
    "density"      = "Density (residents per square km)",
    "log(density)" = "log(Density)"
  ),
  statistic = NULL,
  stars     = FALSE,
  gof_map    = c("nobs", "r.squared"),
  fmt        = 3
)

```

	Linear	Log
Intercept	8.027	-5.290
Density (residents per square km)	0.004	
log(Density)		2.727
Num.Obs.	400	400
R2	0.307	0.392

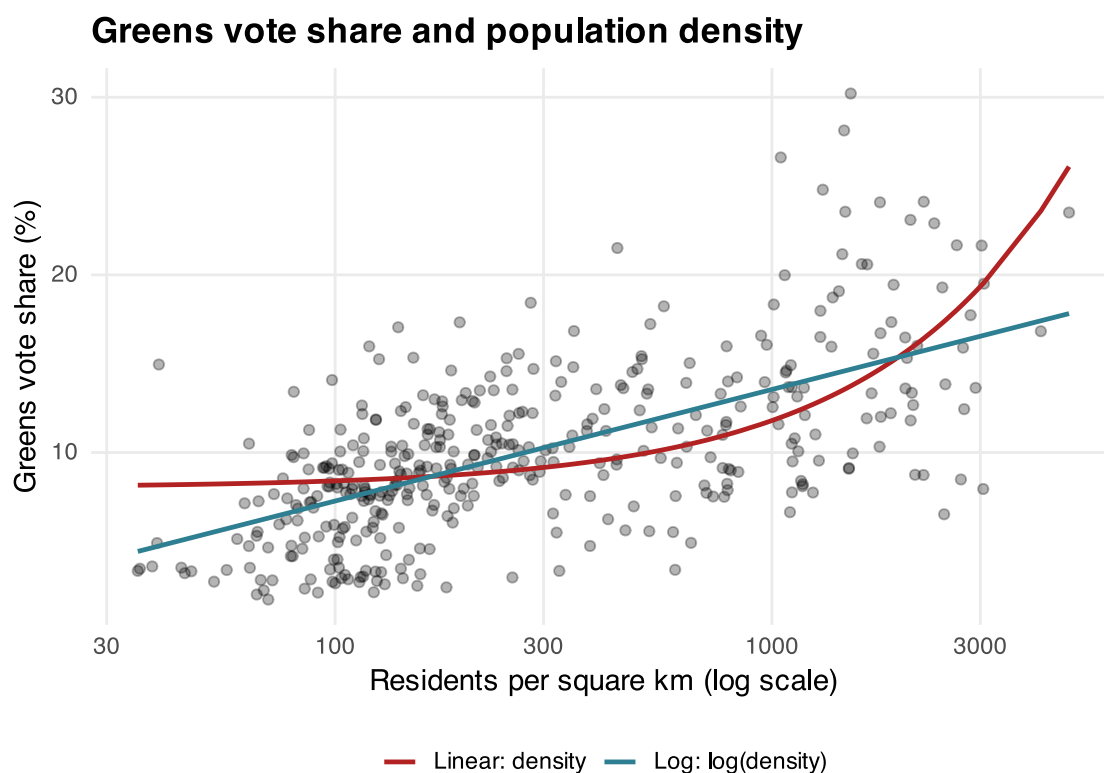
Back on the raw density scale, the log model is a curve — a linear model producing a non-linear relationship between density and Greens support:

```
density_fits |>
  ggplot(aes(x = density)) +
  geom_point(aes(y = greens), data = counties, alpha = 0.3) +
  geom_line(aes(y = .fitted, color = model), linewidth = 1) +
  scale_x_continuous(breaks = seq(0, 4000, 1000)) +
  scale_color_manual(
    values = c("Linear: density" = "firebrick", "Log: log(density)" = course_primary)
  ) +
  labs(
    title = "Greens vote share and population density",
    x      = "Residents per square km",
    y      = "Greens vote share (%)",
    color = NULL
  ) +
  theme_intror()
```



Put the x axis itself on a log scale, and it's the other way round: the log model becomes a straight line, the linear model a curve. Each step to the next gridline now multiplies density by 10:

```
density_fits |>
  ggplot(aes(x = density)) +
  geom_point(aes(y = greens), data = counties, alpha = 0.3) +
  geom_line(aes(y = .fitted, color = model), linewidth = 1) +
  scale_x_log10() +
  scale_color_manual(
    values = c("Linear: density" = "firebrick", "Log: log(density)" = course_primary)
  ) +
  labs(
    title = "Greens vote share and population density",
    x = "Residents per square km (log scale)",
    y = "Greens vote share (%)",
    color = NULL
  ) +
  theme_intror()
```



Interpreting a log coefficient

With a logged predictor, $\hat{\beta}_1$ no longer describes a one-unit step in density, but a *proportional* one. Comparing two counties whose densities differ by a factor c :

$$\Delta\hat{y} = \hat{\beta}_1 \log(cx) - \hat{\beta}_1 \log(x) = \hat{\beta}_1 \log(c)$$

The starting density x drops out: going from 100 to 200 residents per km² changes the prediction exactly as much as going from 1,000 to 2,000.

Doubling a county's density ($c = 2$), with `log_beta_1` from the log model above:

$$\Delta\hat{y} = \hat{\beta}_1 \log(2)$$

```
log_beta_1 * log(2)
```

```
> OUTPUT
```

```
[1] 1.890443
```

A county twice as dense as another is predicted to vote 1.89 percentage points more Green — wherever on the density scale the two counties sit.

A tenfold density ($c = 10$) is one gridline step in the log-axis plot above:

$$\Delta\hat{y} = \hat{\beta}_1 \log(10)$$

```
log_beta_1 * log(10)
```

```
> OUTPUT
```

```
[1] 6.279917
```

A 10% denser county ($c = 1.1$), roughly $\frac{\hat{\beta}_1}{10}$:

$$\Delta\hat{y} = \hat{\beta}_1 \log(1.1) \approx \frac{\hat{\beta}_1}{10}$$

```
log_beta_1 * log(1.1)
```

```
> OUTPUT
```

```
[1] 0.2599426
```

This gives a rule of thumb for a logged predictor: a 10% increase in x goes along with a change of about $\frac{\hat{\beta}_1}{10}$ units in y — here, about 0.273 percentage points of Greens support, close to the 0.260 computed above (the gap comes from $\log(1.1) \approx 0.095$ rather than 0.1).

The same density increase in absolute terms — say, 100 more residents per km² — means a lot in a rural county and almost nothing in a city. That diminishing effect is exactly what the log model captures.

Concluding remarks

You've fitted a regression line by hand, decomposed its fit into TSS, ESS and RSS, reproduced everything with `lm()` and `broom`, and used a log transformation to fit a curved relationship with a linear model. Every model today used a single predictor for all 400 counties at once; whether a relationship holds up once we also account for other differences between counties — East vs. West, for instance — is the question of statistical control in Week 5. The standard errors and p-values in `tidy()`'s output come back in Week 6.

Try it: predicting a county's Greens share

Your task: predict the Greens' vote share in three hypothetical counties with 100, 1,000 and 3,000 residents per km², once with the linear and once with the log density model. Where do the two models disagree most, and why? (`augment()` takes new data through its `newdata` argument.)

```
new_counties <- tibble(density = c(100, 1000, 3000))

linear_predictions <- density_model |>
  augment(newdata = new_counties) |>
  mutate(model = "linear")
log_predictions <- log_density_model |>
  augment(newdata = new_counties) |>
  mutate(model = "log")

bind_rows(linear_predictions, log_predictions) |>
  pivot_wider(names_from = model, values_from = .fitted)
```

➤ OUTPUT

```
# A tibble: 3 × 3
  density linear  log
  <dbl>   <dbl> <dbl>
1    100    8.40  7.27
2   1000   11.8  13.5
3   3000   19.3  16.5
```

Try it: AfD and SPD

Your task: regress the AfD's vote share on the SPD's:

$$\text{AfD}_i = \beta_0 + \beta_1 \text{SPD}_i + \varepsilon_i$$

Compute $\hat{\beta}_0$, $\hat{\beta}_1$ and R^2 by hand, confirm them with `lm()`, and draw the residual plot. How well does SPD support describe AfD support, and what does the residual plot show?

```
afd_spd_hand <- counties |>
  summarize(
    beta_1 = cov(spd, afd) / var(spd),
    beta_0 = mean(afd) - beta_1 * mean(spd),
    r_squared = cor(spd, afd)^2
  )
afd_spd_hand
```

➤ OUTPUT

```
# A tibble: 1 × 3
  beta_1 beta_0 r_squared
  <dbl>   <dbl>   <dbl>
1 -0.834   35.9     0.263
```

```
afd_spd_model <- lm(afd ~ spd, data = counties)
afd_spd_model |>
  tidy()
```

>_ OUTPUT

```
# A tibble: 2 × 5
  term      estimate std.error statistic      p.value
<chr>      <dbl>     <dbl>     <dbl>      <dbl>
1 (Intercept)  35.9        1.16      31.0 2.45e-108
2 spd        -0.834      0.0700    -11.9 3.21e- 28
```

```
afd_spd_model |>
  glance()
```

>_ OUTPUT

```
# A tibble: 1 × 12
  r.squared adj.r.squared sigma statistic      p.value    df logLik   AIC    BIC
    <dbl>      <dbl> <dbl>     <dbl>      <dbl> <dbl> <dbl> <dbl> <dbl>
1   0.263      0.261  7.28     142.      3.21e-28     1 -1361. 2728. 2740.
# i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>
```

```

afd_spd_model |>
  augment() |>
  ggplot(aes(x = spd, y = .resid)) +
  geom_point(alpha = 0.4) +
  geom_hline(yintercept = 0, color = course_primary, linewidth = 0.8) +
  labs(
    title = "Residuals of the AfD-SPD model",
    x      = "SPD vote share (%)",
    y      = "Residual (percentage points)"
  ) +
  theme_intror()

```

